

ECE 3340 Numerical Methods

Homework 2: Prerequisite Programming

Name: *Anaya Scott*

ID: *15991670*

Solve the following problems from **Chapter 2, Prerequisites: Programming**. Use any available space to work out the problem and **place your final solution in the box provided**.

Problem 1: Provide the contents of the registers corresponding to w , x , y , and z

```
...  
unsigned char a = 200;  
unsigned char b = 2;  
unsigned int c = 128;  
unsigned int w = c % 7;  
unsigned int x = a + c;  
unsigned int y = (unsigned char)(a + c);  
unsigned int z = a * b;  
...
```

$$w = \frac{128}{7} \rightarrow 18.2 \rightarrow 128 \% 7 = r = 2 \rightarrow w = 2$$

$$x = 200 + 128 = 328$$

$y = 328$, but in unsigned char is 8 bits which ranges $[0, 255]$

$$\underline{328 - 256 = 72} \rightarrow y = 72$$

this means wrapping around.

$$z = 200 \cdot 2 = 400$$

$w = 2$	$x = 328$	$y = 72$	$z = 400$
---------	-----------	----------	-----------

Problem 2: Allocate space for the parameters of the BLAS sgemm function

The *Basic Linear Algebra Subroutines* (BLAS) library is a popular set of functions for implementing tensor operations. The “gemm” function is a BLAS subroutine that performs a “general matrix-matrix multiply”. Specifically, it evaluates the expression:

$$\mathbf{C} = \alpha \mathbf{AB} + \beta \mathbf{C} \quad \text{where} \quad \mathbf{A} \in \mathbb{R}^{m \times k} \quad \text{and} \quad \mathbf{B} \in \mathbb{R}^{k \times n}$$

The data type for a BLAS function is specified by the first letter. For example, “dgemm” takes arrays representing 64-bit real values and “sgemm” takes arrays representing 32-bit real values. The C signature for the sgemm function looks like this:

```
void sgemm(int m, int n, int k, float alpha, float* A, float* B, float beta, float* C)
```

Write the code to allocate arrays for **A**, **B**, and **C** assuming all of the other values are specified.

```
#include <stdio.h>  
#include <stdlib.h>  
int m, n, k;  
float alpha, beta;  
float* A = (float*) malloc(m * k * sizeof(float));  
float* B = (float*) malloc(k * n * sizeof(float));  
float* C = (float*) malloc(m * n * sizeof(float));
```

Understand
 $A = \mathbb{R}^{m \times k}$
 $B = \mathbb{R}^{k \times n}$
 $C = \mathbb{R}^{m \times n}$